

William Anderson

Senior Software Engineer

Seminole, FL | williamandresondev@outlook.com | +1 (407) 801-1287 |
<https://www.linkedin.com/in/william-andreson-39b88b3aa>

Professional Summary

Senior Software Engineer with 10+ years of experience delivering enterprise software in the **financial services, SaaS, and internal business platform industry**, specializing in **C#, .NET Framework 4.5–4.8, .NET Core 2.1–3.1, .NET 6–8, ASP.NET MVC, ASP.NET Core Web API, and React 16–18** to build secure, scalable, and user-focused systems. Strong background modernizing legacy platforms, fixing production defects, improving API reliability, and shipping new customer-facing features across cloud-hosted environments. Known for translating messy business workflows into maintainable architecture, reducing incident volume, and improving release confidence through strong testing, CI/CD, and observability practices.

Technical Skills

- **Frontend:** React 16–18, TypeScript, JavaScript, ES6+, HTML5, CSS3, Sass, Tailwind CSS, Redux, Context API, React Router, React Query, Vite, Webpack, Responsive UI, component-driven development
- **Backend & APIs:** C#, .NET Framework 4.5–4.8, .NET Core 2.1–3.1, .NET 6–8, ASP.NET MVC, ASP.NET Core Web API, Entity Framework, Entity Framework Core, LINQ, REST APIs, JWT, OAuth2, OpenID Connect, RBAC, background services, microservices, integration services
- **Data & Messaging:** SQL Server, PostgreSQL, MySQL, Redis, MongoDB, Azure Service Bus, RabbitMQ, Elasticsearch
- **Cloud & DevOps:** Azure App Service, Azure Functions, Azure SQL, Azure Storage, Key Vault, Azure DevOps, GitHub Actions, IIS, Docker, Kubernetes basics, AWS exposure, CI/CD pipelines
- **Observability / Quality / Security:** xUnit, NUnit, Jest, React Testing Library, Cypress, Swagger / OpenAPI, Serilog, Application Insights, structured logging, audit logging, rate limiting, OWASP-aligned secure coding

Professional Experience

Viaro Networks Inc.

Senior Software Engineer | January 2021 – December 2025

- Architected and delivered customer-facing and internal workflow platforms using **.NET 6–8, ASP.NET Core Web API, React 17–18, TypeScript, and SQL Server**, helping the business replace fragmented legacy tools with a more maintainable product architecture that supported higher release frequency and cleaner domain boundaries.
- Led a modernization initiative that migrated a large **.NET Framework** codebase to **.NET 6** service-by-service, introducing cleaner dependency injection, standardized API contracts, and staged rollout plans that reduced regression-related deployment issues by **38%** during the transition window.
- Rebuilt several brittle server-rendered modules as **React** interfaces with reusable components, form validation, and role-aware navigation, which improved task completion speed for operations teams by **27%** and reduced UI-related support complaints significantly.

- Resolved a recurring production issue where duplicate submissions created inconsistent transaction records, then implemented idempotency checks, database constraints, and clearer client-side state handling so incident recurrence dropped by **41%** across the affected workflow.
- Designed secure **REST APIs** with **JWT** authentication, audit logging, and permission-aware endpoints, giving product and compliance stakeholders a more trustworthy foundation for handling sensitive operational actions without leaking privileged functionality to the UI.
- Introduced **Redis** caching and query refactoring for heavy reporting screens that had been timing out under peak load, which lowered average response time by **34%** and stabilized performance during monthly usage spikes.
- Built background processing flows for exports, reconciliation jobs, and notification pipelines using hosted services and queue-based patterns, which removed long-running work from synchronous requests and improved perceived application responsiveness for end users.
- Partnered closely with product managers and support teams to convert loosely defined requests into well-scoped deliverables, regularly balancing urgent bug resolution with strategic feature development without letting technical debt accumulate unchecked.
- Improved engineering quality by adding API integration tests, frontend component tests, and release validation steps in **Azure DevOps**, increasing deployment confidence and reducing post-release hotfix activity by **24%** over successive delivery cycles.
- Implemented structured logging with **Serilog** and **Application Insights**, making it easier to trace cross-service failures, identify noisy exceptions, and shorten root-cause analysis time during production incidents that previously required manual guesswork.
- Mentored mid-level developers on **ASP.NET Core**, **Entity Framework Core**, and **React** patterns, helping the team converge on more consistent project structure, simpler service logic, and clearer separation between UI, API, and data access responsibilities.
- Delivered new customer-facing features including approval dashboards, exception handling views, and notification preferences, while keeping backward compatibility for older integrations that the business still depended on during a phased migration strategy.

VividCloud

Software Engineer | January 2016 – December 2020

- Built business-critical web applications with **C#**, **.NET Core 2.1–3.1**, **ASP.NET MVC**, **ASP.NET Core**, **React 16**, and **SQL Server**, supporting internal administration, partner onboarding, and reporting workflows that required stable releases and predictable backend behavior.
- Helped transition selected modules from monolithic MVC pages to API-driven screens with **React**, creating a more flexible frontend layer while still respecting existing release constraints and avoiding risky full rewrites that the team could not justify at the time.
- Fixed a stubborn issue where concurrent edits caused silent data overwrites in shared administrative screens, then added row version checks, better validation feedback, and safer update flows that reduced related data-correction tickets by **32%**.
- Implemented reusable API patterns for pagination, filtering, and standardized error responses, which improved frontend integration speed and reduced inconsistent controller behavior that had been slowing down feature delivery across multiple teams.
- Optimized several slow reporting queries by revisiting indexing strategy, reducing unnecessary joins, and tightening projection logic, which improved page performance by **29%** and made the application noticeably more usable for operations-heavy teams.
- Contributed to a controlled migration from older authentication flows to **OAuth2** and token-based access patterns, improving session handling and strengthening access control for newly exposed API endpoints used by React-based features.
- Delivered notification centers, approval queues, and document-handling workflows that combined backend processing with interactive frontend components, giving business users a smoother experience for exception-heavy tasks that had previously been handled through email and spreadsheets.

- Reduced production debugging time by introducing structured logging, correlation IDs, and clearer exception handling around external integrations, making it easier to isolate whether failures originated in business logic, database access, or partner APIs.
- Supported CI/CD improvements in **Azure DevOps** by standardizing build steps, test execution, and deployment checks, which improved release consistency and reduced environment-specific surprises that had previously delayed production rollouts.
- Worked flexibly across frontend, backend, database, and deployment tasks depending on sprint pressure, which made it possible to unblock releases quickly and helped the team maintain delivery momentum during periods of changing requirements.
- Collaborated with QA and product stakeholders to validate bug fixes and edge cases thoroughly, especially for sensitive financial and approval workflows where seemingly small defects could create downstream reconciliation effort and user distrust.

Minimalist Moon

Software Developer | *January 2013 – December 2015*

- Developed internal web platforms using **C#, .NET Framework 4.5–4.7, ASP.NET MVC, JavaScript, jQuery**, and **SQL Server**, building practical business features at a time when stable server-rendered applications still formed the core of many operational systems.
- Implemented modules for user administration, approvals, reports, and operational tracking, focusing on maintainable controller and service logic so teams could process daily work with fewer manual corrections and less dependency on developer intervention.
- Solved a recurring defect where large report requests locked the session and encouraged repeated user actions, then moved heavy processing into scheduled routines and added clearer status handling to improve reliability for support-heavy workflows.
- Refactored brittle data-access code and inefficient SQL used by listing pages, improving query performance and reducing load times by **31%** on several commonly used screens that had gradually become slower as data volume increased.
- Added early API-style endpoints and lightweight interactive widgets for selected screens as the team began moving toward more dynamic UI patterns, creating a lower-risk foundation for later frontend modernization without forcing a full platform rewrite.
- Implemented change-history tracking for sensitive record updates, making it easier for support teams to investigate who changed statuses, ownership, and approvals when production issues surfaced and accountability became important.
- Improved validation and error messaging in CSV import flows so bad records could be corrected faster by business users, which increased first-pass import success by **23%** and reduced repeated back-and-forth between operations and engineering.
- Contributed bug fixes across backend, UI, and deployment scripts whenever needed, showing flexibility in a smaller team environment where solving production issues quickly mattered more than rigid ownership boundaries.
- Strengthened release reliability by cleaning up environment-specific configuration, reducing manual deployment mistakes, and documenting recurring setup problems that had previously slowed onboarding and hotfix preparation.
- Worked closely with senior engineers to improve code review quality, testing discipline, and naming consistency, which helped raise maintainability standards and prepared the system for gradual modernization in later years.

Microsoft

Software Developer Intern | *July 2012 – December 2012*

- Supported enterprise web application development using **C#, ASP.NET MVC, JavaScript, jQuery**, and **SQL Server**, contributing to feature work and defect resolution while learning how large engineering teams structured delivery, review, and testing practices.

- Implemented small but meaningful enhancements for administrative screens and internal tooling, carefully aligning frontend behavior with backend validation rules so the user experience stayed predictable even when requirements evolved mid-sprint.
- Investigated and fixed UI defects related to inconsistent field validation and state handling, improving form reliability and reducing repeat bug reports from internal users who depended on those screens for daily operational tasks.
- Assisted with controller, service, and query updates under guidance from senior engineers, gaining practical experience in how production-safe changes were designed, reviewed, and tested before being allowed into shared environments.
- Contributed to regression testing and issue verification efforts after release candidates, helping catch edge cases earlier and reinforcing the importance of disciplined quality checks in environments where even minor regressions could slow larger teams.
- Documented recurring setup and troubleshooting steps for local development environments, which helped newer team members become productive faster and reduced repeated interruptions for simple onboarding and configuration questions.
- Participated in code reviews and technical discussions that emphasized maintainability, secure coding, and readability, shaping a strong foundation in professional engineering habits that carried into later full-time platform work.
- Learned to work across backend logic, database access, and UI bug fixing in a practical way, demonstrating flexibility and curiosity rather than staying limited to a single layer of the stack during the internship experience.
- Delivered assigned tasks consistently and responded well to feedback, building trust with mentors and showing early ability to adapt quickly to team standards, production expectations, and real-world software delivery constraints.

Education

Florida State University

Bachelor's Degree in Computer Science | 2008 – 2012